TP8 — Food Store en la feria

Informe: 1.a, 2.c, 3.a, 3.b y 3.c

Base de Datos II · Unidad 8 · Datos inventados sobre una copia de prueba.

1.a Qué datos viajan a la tablet

Viaja	No viaja
`id` del producto	`usuario` (nombre, apellido, mail, celular)
`nombre`	`contrasena`
`precio_centavos`	pedidos históricos (`pedido` , `detalle_pedido`)
`stock_visto`	totales y datos de pago
`disponible` / `eliminado`	`created_at` del servidor
`fecha_copia`	logs y auditoría

Viaja solo lo mínimo para que el vendedor pueda mostrar el menú y armar un borrador: identidad del producto, precio y stock. No viajan datos personales porque la tablet se pierde o se roba más fácil que el servidor, y no hay razón para que el dispositivo tenga el mail o la clave de un cliente.

La fuente de verdad es PostgreSQL. La tablet guarda una copia de solo lectura: si su precio o su stock difieren del servidor, gana el servidor.

2.c Casos del semáforo

Cada caso se armó con un borrador en SQLite, un cambio en la copia de PostgreSQL, una instantánea nueva y la corrida de `semaforo.sql` .

Caso	Qué se preparó	Resultado
C2	Borrador del producto 2 a 265787 centavos; en el servidor se subió el precio a 266287	`CONFLICTO — cambio el precio: 2`
C2 (acepta)	Se actualizó el precio del borrador y se creó la solicitud `S-C2b`	`SIMULADO_APROBADO — todo coincide`
C3	Borrador de 10 unidades del producto 4 (stock visto: 3)	`RECHAZADO — stock insuficiente: 4`
C4	Baja lógica del producto 5 en el servidor	`RECHAZADO — dado de baja: 5`
C4 (recarga)	Se recargó `producto_cache` desde la instantánea nueva	La fila queda con `eliminado = 1` , pero no aparece en el catálogo disponible
C5	Borrador del producto 1 sin ningún cambio	`SIMULADO_APROBADO — todo coincide`
C6	Se reenvió la solicitud `S-C5` (mismo UUID)	`UNIQUE constraint failed: cola_sync.solicitud_uuid`

Estado final de la cola:

```
S-C2   CONFLICTO          cambio el precio: 2
S-C2b  SIMULADO_APROBADO  todo coincide
S-C3   RECHAZADO          stock insuficiente: 4
S-C4   RECHAZADO          dado de baja: 5
S-C5   SIMULADO_APROBADO  todo coincide
```

Explicación de cada uno

- **C2**: el precio visto ya no coincide con el del servidor, entonces es amarillo. Al aceptar el cliente se actualiza el borrador y la solicitud nueva vuelve a verde. La solicitud vieja queda en `CONFLICTO`: no se reescribe.
- **C3**: la cantidad pedida supera el stock de la instantánea. Es rojo y no se evalúa el precio, porque el pedido no se podría cumplir igual.
- **C4**: la baja es lógica, así que la fila sigue en `producto_cache` con `eliminado = 1`. La consulta del catálogo la filtra, por eso "desaparece" sin borrar nada.
- **C5**: precio, stock y disponibilidad coinciden. Es verde, y **no es una venta**: no se escribió nada en PostgreSQL.
- **C6**: la clave primaria sobre `solicitud_uuid` impide el duplicado, y el semáforo solo mira `PENDIENTE`, así que repetir la simulación no reevalúa.

Protección real (parada 8). En un sistema de verdad la idempotencia no se resuelve en la tablet: el servidor recibe la solicitud con su UUID, la busca en una tabla de solicitudes procesadas dentro de la misma transacción que confirma la venta, y si ya existe devuelve el resultado anterior sin volver a aplicarla. La tablet solo reintenta; el servidor decide.

El servidor no cambió

	Antes	Después
pedidos	7	7
detalles	10	10
stock total	5.007.341	5.007.341

El único cambio (baja del producto 5) lo hizo la preparación del caso C4, no la simulación. La tablet no confirmó ninguna venta ni descontó stock.

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda sobre esos datos
PostgreSQL	catálogo real (`producto` , `categoria`), pedidos (`pedido` , `detalle_pedido`), usuarios	el **servidor**. Es la única fuente de verdad de precio, stock y ventas
SQLite (tablet)	copia del catálogo (`producto_cache`), borradores (`pedido_borrador` , `item_borrador`) y cola de envío (`cola_sync`)	la **tablet**, pero solo sobre borradores. Nunca sobre precio, stock ni ventas
MongoDB (Unidad 7)	pedidos migrados de PostgreSQL a la colección `pedidos` : un documento por pedido, con el detalle embebido en un array (`usuario` y `producto` solo por id). Se usó para comparar los dos motores	**nadie**: es una copia de estudio. La escribió el script de migración una sola vez, se consultó de solo lectura y no recibe altas de la feria

Regla que atraviesa los tres: **cada dato tiene un solo dueño**. El catálogo y las ventas son de PostgreSQL; la tablet es una copia descartable; MongoDB es una copia de estudio que no participa del circuito de venta, así que no puede bloquear ni contradecir al servidor.

Por qué MongoDB no entra en el circuito. En la Unidad 7 migramos `pedido` y `detalle_pedido` a documentos embebidos para medir la diferencia: leer un pedido completo en Mongo es una sola lectura, sin `JOIN`, mientras que

en PostgreSQL son varias tablas. Pero la coherencia que exige la feria (`subtotal = cantidad × precio_unitario`, `total` que cierra, stock que no se vaya a negativo) en Mongo quedaría a cargo de la aplicación, no del motor. Por eso el TP7 concluyó que **PostgreSQL sigue siendo la fuente de verdad** y Mongo quedó como análisis, no como parte del sistema.

3.b Plan de mantenimiento

¿Qué se respalda y cómo se prueba la restauración? Se respalda `food_store_test` completo con `pg_dump` (esquema + datos), todos los días al cierre y antes de cada cambio de esquema, con retención de 7 días y una copia fuera de la máquina. La restauración se prueba una vez por mes en una base aparte (`food_store_restore`): se restaura el dump, se corre `docs/sql/verificar_servidor.sql` y se comparan pedidos, detalles y stock total contra el respaldo. Si los números no coinciden, el respaldo no sirve.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? Nada automático: la tablet no escribe. Al reconectar, descarta su instantánea y baja una nueva. Las solicitudes `PENDIENTE` se reevalúan contra esa instantánea y muchas van a pasar a `CONFLICTO` o `RECHAZADO`, porque el respaldo puede tener precios viejos o stock que ya no existe. Nunca se aplican a ciegas.

¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO? Al cierre de cada feria revisa el encargado de operaciones; el dueño hace una pasada semanal. Los `CONFLICTO` se resuelven con el cliente en el momento (acepta el precio nuevo o se cancela) y los `RECHAZADO` se archivan con su motivo, que ya queda guardado en `cola_sync.motivo`.

3.c Code review con IA

Se revisaron los scripts de la cursada con el agente de OpenCode. Cada hallazgo se comprobó ejecutando algo contra la base, no leyendo el código.

#	Unidad	Hallazgo	Decisión	Motivo
1	U8	`pruebas_sqlite.sql` empieza con `DELETE FROM producto_cache`, así que corrido sobre la base entregada borra la instantánea	**Corregido**	Se documentó y se corre sobre una copia
2	U8	`semaforo.sql` usa `snapshot_servidor`, pero `schema_sqlite.sql` no crea esa tabla	**Corregido**	Se agregó el paso de importación al README
3	U2	`CREATE INDEX idx_producto_stock ON producto(stock DESC)` usa `DESC`, que en PostgreSQL es redundante	**Descartado**	El `DESC` no aporta ni perjudica: el plan es el mismo
4	U1	`usuario.contrasena` guarda la clave en texto plano	**Aceptado**	Es una deuda real, pero el esquema del servidor es de otra unidad y no se toca en el TP8

Cómo se comprobó cada uno

1. **Pruebas borra la caché.** Sobre una copia de `foodstore_movil.db`: `SELECT COUNT(*) FROM producto_cache` dio **50.001** antes y **6** después de correr `pruebas_sqlite.sql`.
2. **Falta `snapshot_servidor`.** Se creó una base desde cero con `schema_sqlite.sql` y se corrió `semaforo.sql`: `Parse error: no such table: snapshot_servidor`.

3. **El `DESC` es redundante.** Se creó el índice en orden ascendente y se pidió el plan:

```
CREATE INDEX idx_tmp_stock ON producto (stock) WHERE eliminado = FALSE;
EXPLAIN SELECT * FROM producto WHERE eliminado = FALSE ORDER BY stock DESC LIMIT 100;

Limit
  ->  Index Scan Backward using idx_tmp_stock on producto
```

   PostgreSQL recorre el B-Tree al revés, así que `DESC` no cambia el plan. Se descarta el cambio: el índice original funciona igual.

4. **Contraseñas en texto plano.**

```
SELECT id_usuario, mail, contrasena FROM usuario LIMIT 3;
-- 1 | juan@mail.com  | 1234
-- 2 | maria@mail.com | 1234
```

   Se acepta como deuda: corregirlo implica migrar el esquema y los datos de todas las unidades anteriores, y no es el alcance de este TP.

Cierre

- La tablet nunca modificó precio, stock ni ventas del servidor.
- `SIMULADO_APROBADO` no es una venta: es una simulación que no escribe en PostgreSQL.
- Todo lo generado con IA se ejecutó y se comprobó antes de darlo por bueno.